Beyond Accuracy@k: How Localization Quality Shapes Repository-Level LLM Program Repair

Juan Javier Samuel Z (东万) · L25020002 · October 2026

1. Context

Resolving real GitHub issues across whole codebases has become the main testbed for LLM-based automated program repair (APR), with SWE-bench [3] as its reference benchmark. A recurring finding is that the hardest step happens before any patch is written: the model must first find *where* to edit. MAGIS's empirical study [2] concludes that "locating lines is a key factor for GitHub issue resolution" and that "increased complexity, particularly in terms of the number of files and functions changed, tends to hinder issue resolution." The latest LLM4APR systematic review [1] likewise states that "fault localization is a precondition of patch generation" (§9.3.2). The step is also costly: SHERLOC [7] reports that repair agents spend about half of their budget locating faults before editing. We therefore take localization, the first bottleneck MAGIS identifies, as the unresolved bottleneck examined in this statement.

2. What Prior Work Has Done

Issue localization is therefore studied as a task in its own right. Agentless [10] narrows hierarchically from files to functions to lines; LocAgent [4], CoSIL [5] and SweRank [6] use code graphs, iterative call-graph search and retrieve-and-rerank, respectively; SHERLOC [7] outputs structured diagnoses instead of bare locations. These methods are judged mainly by retrieval metrics (Acc@k, Recall@k), and their value for repair is reported as end-to-end gains: adding CoSIL to Agentless, for example, raises the resolution rate by 2.98–30.5% [5]. Loc2Repair [8] was the first to separate the two steps under matched conditions: on SWE-bench Verified, predicted file-level localization lifted the pooled resolved rate from 44.7% to about 49%, and gold localization lifted it to 52.4%. At the function level, the localization assumption has been examined more directly. RepairAgent [11] adopts perfect fault localization by default, and a recent survey counts 10 of 66 LLM-based repair systems that explicitly assume it [14]. D4C [12] drops statement-level localization but still assumes the buggy function is given, and reports that letting the LLM refine the whole function is more effective than the localize-hunks-then-repair workflow. Campos et al. [13] find that replacing perfect function-level localization with the top-3 functions from a spectrum-based localizer lowers accuracy by 7.83–16.20% on a Defects4J subset.

3. The Gap

These function-level studies vary localization only as perfect versus automated, on single-function Java bugs. For repository-level issues, where the localization output is a *set of candidate files* handed to an agent, we still know little about how the shape of that set affects whether repair succeeds. Three questions remain open. **(i) Precision versus recall.** In the SWE-bench study [3], as analyzed by MAGIS [2], a larger retrieval budget raised file recall from 29.58% to 51.06% but *lowered* Claude-2's resolved ratio from 1.96% to 1.22%. The latest LLM-APR survey likewise warns that retrieval can miss key files while extra context can exceed model limits [14]. Yet current evaluations compare only a few discrete conditions (none, predicted, gold) [8], so we cannot tell whether a localizer should optimize Acc@1 or Recall@5. **(ii) Interaction with change complexity.** Multi-file changes are harder [2], and some issues remain unresolved even with gold localization [8], but no study we reviewed tests whether localization quality matters more for multi-file than for single-file fixes. **(iii) Genuine correctness.** All of the gains above are measured by passing the benchmark's tests, yet strengthened tests reject 19.71% of previously passing patches from the top-30 SWE-bench Verified agents [9]. Whether better localization yields correct patches, or merely more *plausible* ones, is unknown. The LLM4APR research agenda calls for cost-aware evaluation (§9.2.2) and treats localization and validation as interconnected (§9.3.2) [1], but it does not answer these questions for repository-level repair.

4. Why It Matters and Proposed Study

Closing this gap matters for both research and practice. For researchers, it shows which localization metric actually predicts repair success, so that localizer design stops optimizing retrieval scores that may not transfer. For practitioners, localization consumes about half of an agent's budget [7], so knowing how much localized context is "enough" directly informs the cost–quality trade-off. We propose a controlled empirical study on SWE-bench Verified that keeps the backbone LLM and the agent scaffold fixed while manipulating the localized context the agent receives (adding distractor files to the gold file set, or removing gold files from it), and that stratifies instances by change complexity following MAGIS's regression analysis [2]. Unlike Loc2Repair [8], which compares fixed localization conditions, and the function-level studies [11–13], this design varies localization precision and recall at the repository level and checks correctness beyond test-passing. The study is feasible with public benchmarks, an open-weight model (e.g., Qwen2.5-Coder-32B, as used in [5]), and a stratified subset of instances to bound cost.

5. Research Questions

RESEARCH QUESTION	INDEPENDENT VARIABLE(S)	DEPENDENT VARIABLE(S)
RQ1. How do the precision and recall of the file-level localization given to an LLM repair agent affect its resolved rate and cost on repository-level issues?	Localization precision {1.0, 0.5, 0.25} (via added distractor files) × recall {1.0, ≈0.5} (keeping half of the gold files, rounded up; exact value logged per instance). Recall is varied only on multi-file instances, since a single-file instance allows recall of only 0 or 1. Controlled: backbone LLM, scaffold, prompt, instance set.	Resolved rate (%); input/output tokens and USD cost per instance.
RQ2. Does the effect of localization quality on repair success depend on the complexity of the required change?	Localization condition {none, predicted (e.g., CoSIL [5]), gold}. Moderator: gold-patch complexity (single- vs. multi-file; number of hunks).	Resolved rate, analyzed by logistic regression with an IV × complexity interaction term (extending [2]).
RQ3. Does diagnostic localization (location plus a natural-language fault explanation) produce more genuinely correct patches than location-only output?	Localization output form {file path; file + function; diagnostic finding as in [7]}.	Resolved rate on original tests; correctness rate under strengthened tests [9] (or manual validation of a sample); overfitting rate (patches that pass the original tests but fail the strengthened ones).

References

[1] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, Z. Chen. A Systematic Literature Review on Large Language Models for Automated Program Repair. *ACM TOSEM*, 2026. arXiv:2405.01466v4.

[2] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, Y. Cheng. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. *NeurIPS*, 2024. arXiv:2403.17927.

[3] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.

[4] Z. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. *ACL*, 2025. arXiv:2503.09089.

[5] Z. Jiang, X. Ren, M. Yan, W. Jiang, Y. Li, Z. Liu. Issue Localization via LLM-Driven Iterative Code Graph Searching (CoSIL). *ASE*, 2025. arXiv:2503.22424.

[6] R. G. Reddy, T. Suresh, J. Doo, et al. SweRank: Software Issue Localization with Code Ranking. *ICLR*, 2026. arXiv:2505.07849.

[7] H. Tamoyan, S. Narenthiran, E. Arakelyan, M. Mezini, B. Ginsburg. SHERLOC: Structured Diagnostic Localization for Code Repair Agents. arXiv:2606.24820, 2026.

[8] M. N. Al Awad, S. Ivanov. Loc2Repair: A Framework for Evaluating the Impact of File-Level Issue Localization in Repo-Level LLM Repair. *GeCoIn Workshop @ IJCAI-ECAI*, 2026. arXiv:2606.30963.

[9] B. Yu et al. SWE-ABS: Adversarial Benchmark Strengthening Exposes Inflated Success Rates on Test-based Benchmark. arXiv:2603.00520, 2026.

[10] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.

[11] I. Bouzenia, P. Devanbu, M. Pradel. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. arXiv:2403.17134, 2024.

[12] J. Xu, Y. Fu, S. H. Tan, P. He. Aligning the Objective of LLM-based Program Repair. *ICSE*, 2025. arXiv:2404.08877.

[13] V. Campos, R. Shariffdeen, A. Ulges, Y. Noller. Exploring Generalizable Automated Program Repair with Large Language Models. arXiv:2506.03283, 2025.

[14] B. Yang, Z. Cai, F. Liu, B. Le, L. Zhang, T. F. Bissyandé, Y. Liu, H. Tian. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. arXiv:2506.23749, 2025.